



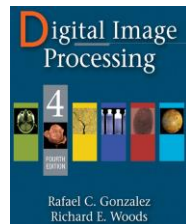
دانشگاه اصفهان

دانشکده مهندسی کامپیوتر

گزارش پروژه نهایی - تصویرپردازی رقمی

ردیابی ویدئویی با واژگان باز

توسط SAM3



پدیدآورنده:

محمد امین کیانی

۴۰۴۳۶۴۴۰۰۸

دانشجوی کارشناسی ارشد، دانشکده‌ی کامپیوتر، دانشگاه اصفهان، اصفهان،
Aminkianiworkeng@gmail.com

استاد درس: دکتر نقش نیلچی

نیمسال دوم تحصیلی ۱۴۰۴-۰۵

فهرست مطالب

مستندات.....	۳
۰- مسئله و تحلیل کلی آن:.....	۳
۱- مبانی مدل Segment Anything Model 3:.....	۱۲
۲- پیاده‌سازی مراحل انجام پروژه:.....	۱۴
۳- ارزیابی نتایج:.....	۲۹
۴- منابع.....	۳۳

مستندات

۰- مسئله و تحلیل کلی آن:

عنوان پروژه: ردیابی ویدئویی با واژگان باز (Open-Vocabulary Video Tracking) با معماری SAM3 و استخراج مشخصه‌های هندسی حرکت.

مقدمه و بیان مسئله

در سیستم‌های سنتی بینایی ماشین، ردیابی اشیاء محدود به کلاس‌های از پیش تعریف‌شده (مانند انسان، خودرو و غیره) بود. با ظهور مدل‌های پایه‌ای بینایی (Vision Foundation Models) مانند سامسونگ/متا، امکان دسته‌بندی و قطعه‌بندی تصاویر با پرامپت‌های متنی و به صورت بدون ارسال مثال (Zero-Shot) فراهم شده است. در این پروژه، هدف ردیابی یک شیء پویای تعریف شده توسط زبان طبیعی در طول توالی فریم‌های یک ویدیو، استخراج ماتریس حرکت آن و رسم هندسی مسیر تغییرات است.

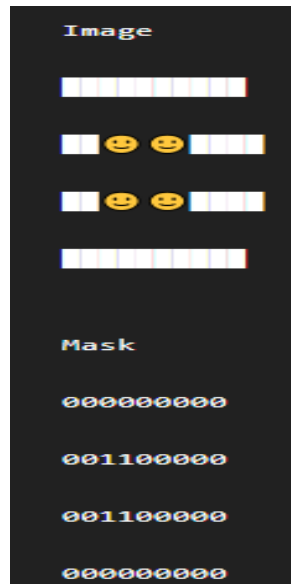
تئوری ریاضی و الگوریتم محاسباتی

الف) قطعه‌بندی و انتقال ماسک (Mask Propagation)

معماری SAM3 با تلفیق دکودر تصویر و ویژگی‌های حافظه مکانی-زمانی، ماسک اولیه تولید شده توسط پرامپت متنی در فریم t_0 را به فریم‌های آتی t_n اعمال می‌کند. تابع بهینه‌سازی مدل بر مبنای تشابه ویژگی‌های کانولوشنی و ترانسفورمری استخراج شده از شیء هدف عمل می‌نماید.

در واقع ماسک یعنی:

SAM3 می‌گوید: "من فکر می‌کنم این قسمت شیء است". پس یک تصویر سیاه و سفید می‌سازد که به آن Binary Mask می‌گوییم که ماتریسی از ۰ و ۱ هاست و ۱ یعنی وجود شیء و ۰ یعنی پس‌زمینه است. (مثل یک ماسک عمل می‌کند و فقط همان قسمت‌های سفید انتخاب می‌شوند انگار که روی تصویر یک شابلون گذاشته باشیم).



سپس مطابق تمرین دوم که تحویل دادیم، عملیات مورفولوژی (ریخت‌شناسی) را انجام دادیم تا نویزگیری و اصلاحات مورد نیاز را بر روی ورودی انجام دهیم.

در مرحله‌ی بعدی **propagate** انجام می‌دهیم که آن هم یعنی انتشار یا گسترش اطلاعات از یک فریم به فریم‌های بعدی. در SAM۳ ابتدا در فریم اول شیء با متن شناسایی می‌شود، سپس مدل با استفاده از ویژگی‌های بصری و حافظه زمانی (Temporal Memory)، ماسک همان شیء را در فریم‌های بعدی پیش‌بینی می‌کند و دیگر لازم نیست در هر فریم دوباره متن را وارد کنیم. این فرآیند در API جدید با درخواست `propagate_in_video` انجام می‌شود.

ب) استخراج مرکز ثقل (Centroid Calculation)

پس از استخراج ماسک باینری $I(x,y)$ برای هر فریم، از گشتاورهای تصویر برای به‌دست آوردن مرکز ثقل هندسی استفاده می‌شود. طبق مباحث فصل‌های اولیه کتاب پردازش تصویر دیجیتال گنزالس، گشتاور مرتبه $(p+q)$ یک تصویر دوبعدی گسسته به صورت زیر تعریف می‌شود:

$$M_{pq} = \sum_x \sum_y x^p y^q I(x, y)$$

که در آن $I(x,y)$ مقدار پیکسل در مختصات (x,y) است (برای ماسک باینری، مقدار ۱ برای شیء و ۰ برای پس زمینه است). گشتاور مرتبه صفر $M_{\{..\}}$ نشان دهنده مساحت کل شیء قطعه بندی شده است. مختصات مرکز ثقل هم از روابط زیر محاسبه می گردد:

- ابتدا **Moment صفر** را حساب می کنیم (M_{00}) که برابر است با تعداد کل پیکسل های سفید یعنی $\sum I(x,y) = M_{00}$.

- سپس **Moment اول در جهت X** را حساب می کنیم. M_{10} یعنی هر مختصات $x \times$ مقدار آن پیکسل جمع شوند. ($\sum xI(x,y) = M_{10}$) و همین مومن مرتبه اول را در جهت Y حساب می کنیم که به صورت $\sum yI(x,y) = M_{01}$ می شود و در نهایت:

$$\bar{x} = \frac{M_{10}}{M_{00}}, \quad \bar{y} = \frac{M_{01}}{M_{00}}$$

که همان مرکز جرم تصویر است و Moment همان "گشتاور" توزیع پیکسل هاست و اطلاعاتی درباره ی مساحت، مرکز جرم و شکل جسم به ما می دهد.

پس مرکز جرم (Centroid) یعنی:

یک نقطه وجود دارد که صفحه روی آن تعادل کامل دارد و آن نقطه را Center of Mass یا Centroid می گویند که همیشه مرکز به سمت قسمت بزرگ تر کشیده می شود و همین تفاوت مهم Centroid و Bounding Box Center است.

اما مرکز یک ناحیه (Bounding Box Center) یعنی:

فرقی ندارد داخل آن چه شکلی باشد و به سمت قسمت بزرگ تر کشیده نمی شود (جرم موثر نیست بلکه وسط ناحیه مهم است). پس مرکز آن مختصات $x+w/2$ و $y+h/2$ است ولی Centroid داخل خود شکل را نگاه می کرد.

اگر Area کوچک باشد، Moment را حساب نمی کنیم زیرا:

به فرض مثلاً فقط دو پیکسل باقی مانده باشد، مرکز جرم این دو پیکسل هیچ ربطی به انسان ندارد و کاملاً تصادفی است. پس خروجی Centroid می‌پرد و اگر $Area < 150$ باشد، باید از Bounding Box Center استفاده کنیم چون Bounding Box را خود SAM^۳ با اطمینان بیشتری پیش‌بینی کرده و در این شرایط پایدارتر است.

برای ارزیابی نیز معیار Confidence را انتخاب کردیم:

که یعنی با احتمال چند درصد مطمئنیم این شیء انتخابی همان شیء است (کیفیت تشخیص) و عدد حاصل را Confidence یا Score می‌گویند.

برای انتخاب فیلترها (Moving Average، Median و Savitzky–Golay) نیز:

Moving Average (میانگین متحرک)

پس خطا (عدد پرت و دور) را هم وارد میانگین می‌کند.

Median Filter (میان)

چون مقدار وسط را انتخاب می‌کند، به داده‌های پرت حساس نیست و به همین دلیل برای حذف پرش‌های لحظه‌ای مرکز جرم بسیار مناسب است.

Savitzky–Golay

این فیلتر به جای میانگین گرفتن، روی چند نقطه یک چندجمله‌ای (مثلاً درجه ۲ یا ۳) برازش می‌کند و مقدار هموار (Smooth) را از روی آن به دست می‌آورد. در نتیجه شکل کلی حرکت، شیب‌ها و انحنای را بهتر حفظ می‌کند و نسبت به میانگین ساده، جزئیات بیشتری را نگه می‌دارد.

چرا در پروژه ما فیلتر Median انتخاب شد؟

چون مسیر ردیابی گاهی دارای پرش‌های ناگهانی ناشی از خطای مدل بود و Median این پرش‌ها را به خوبی حذف می‌کند، در حالی که میانگین متحرک ممکن است خود تحت تأثیر همان پرش قرار گیرد. البته اگر هدف تحلیل دقیق حرکت فیزیکی با حفظ انحنایها باشد، Savitzky–Golay گزینه‌ی بهتری است؛ اما برای این پروژه‌ی ردیابی ویدئویی، Median انتخابی ساده، پایدار و بسیار مناسب است و طبق آزمایش و نتایج، بهتر از بقیه عمل کرد.

```
from scipy.signal import savgol_filter

trajectory["x_smooth"] = savgol_filter(
    trajectory["x"],
    window_length=9,
    polyorder=2,
    mode="nearest"
)

trajectory["y_smooth"] = savgol_filter(
    trajectory["y"],
    window_length=9,
    polyorder=2,
    mode="nearest"
)

trajectory["x_smooth"] = trajectory["x"].rolling(
    7,
    center=True,
    min_periods=1
).median()

trajectory["y_smooth"] = trajectory["y"].rolling(
    7,
    center=True,
    min_periods=1
).median()
```

ساختار پروژه و خروجی‌های مورد انتظار

- **خروجی متنی (CSV):** یک ماتریس داده شامل ستون‌های Frame، X و Y تولید شود که تغییرات آنی مرکز جرم شیء را در مقیاس پیکسل ثبت کند. این داده‌ها برای تحلیل‌های کینماتیکی (شتاب و سرعت شیء) در پله‌های بعدی کاربرد دارند.
- **خروجی نموداری (PNG):** نمودار دو بعدی مسیر حرکت، سیستم مختصات تصویر (مبدأ در بالا سمت چپ) را شبیه‌سازی کند و الگوهای رفتاری شیء از نقطه شروع (Start) تا پایان (End) را با وضوح بالا به تصویر بکشد.

- **ویدیو (MP4):** ماسک‌های پیش‌بینی شده با ترکیب رنگ سبز و شفافیت ۳۰٪ روی ویدیوی اصلی نگاشت شوند که بیانگر دقت بالای مدل در حفظ مرزهای شیء بدون وقوع پدیده شکست ردیابی (Tracking Drift) باشد.

وینشو
↓
SAM3
↓
Mask
↓
حذف نویز
↓
انتخاب بزرگترین ناحیه
↓
اگر ناحیه خوب بود
↓
Moments
↓
Centroid
↓
اگر ناحیه بد بود
↓
Center Bounding Box
↓
نخیره مختصات
↓
Smooth
↓
Trajectory
↓
Output Video

راهنمای توکن هگینگ فیس (Hugging Face)

۱. چرا باید توکن بسازیم؟

مدل‌های پیشرفته مانند SAM3 (Segment Anything Model 3) به دلیل حجم بالا و قوانین لایسنس شرکت Meta، وزن‌های رسمی خود (Checkpoints) را در ریپازیتوری‌های بسته یا محافظت‌شده‌ی Hugging Face قرار می‌دهند. برای اینکه کد بتواند در بستر کولب یا لوکال به صورت خودکار این وزن‌ها را دانلود کند، باید هویت ما احراز شود.

۲. کدام نوع توکن را بسازیم؟

هگینگ فیس سه نوع توکن ارائه می‌دهد:

Read (خواندنی)، Write (نوشتنی) و Fine-grained (دسترسی محدود شده).

• نوع مورد نیاز ما : Read

- دلیل: پروژه فقط نیاز به دانلود (Download/Fetch) وزن‌های مدل دارد و قرار نیست هیچ فایلی را روی سرورهای هگینگ فیس آپلود کند یا تغییر دهد. بنابراین توکن ساده‌ی Read بالاترین امنیت و کارایی را برای ما دارد.

۳. روش ساخت و استفاده:

۱. وارد سایت [Hugging Face](https://huggingface.co) شده و یک حساب کاربری می‌سازیم.
۲. به بخش Settings و سپس Access Tokens می‌رویم.
۳. روی New Token کلیک کرده، نامی برای آن انتخاب کرده و نوع آن را روی Read می‌گذاریم.
۴. توکن ساخته شده (که با hf_ شروع می‌شود) را کپی کرده و در کد جایگذاری می‌کنیم.

```
from huggingface_hub import login

login("hf_XXXXXXXXXXXXXXXXXXXXXXXXXXXX")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_http.py in hf_raise_for_status(response, endpoint_name)
760     try:
--> 761         response.raise_for_status()
762     except httpx.HTTPStatusError as e:

/usr/local/lib/python3.12/dist-packages/httpx/_models.py in raise_for_status(self)
828         message = message.format(self, error_type=error_type)
--> 829         raise HTTPStatusError(message, request=request, response=self)
830

HTTPStatusError: Client error '403 Forbidden' for url 'https://huggingface.co/facebook/sam3/resolve/3c879f39826c281e95690f02c7821c4de09afae7/.gitattributes'
For more information check: https://developer.mozilla.org/en-US/docs/Web/HTTP/Status/403

The above exception was the direct cause of the following exception:

GatedRepoError                                Traceback (most recent call last)
/tmp/ipykernel_3921/3359763388.py in <cell line: 0>()
      1 from huggingface_hub import snapshot_download
      2
----> 3 snapshot_download(
      4     repo_id="facebook/sam3",
      5     local_dir="sam3_weights"
      ...

GatedRepoError: 403 Client Error. (Request ID: Root=1-6a3dc39a-3928ceab441b1253556a0233;54448a38-8707-4cfc-a004-77b34b0287ef)

Cannot access gated repo for url https://huggingface.co/facebook/sam3/resolve/3c879f39826c281e95690f02c7821c4de09afae7/.gitattributes.
Access to model facebook/sam3 is restricted and you are not in the authorized list. Visit https://huggingface.co/facebook/sam3 to ask for access.
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

اما به علت محدودیت‌ها و تحریم مجبور هستیم که توکن این سایت را استفاده نکنیم زیرا اجازه‌ی دسترسی نمی‌دهد و نمی‌توان کدی را بر روی این مدل اجرا کرد. چون مخزن رسمی SAM^۳ یک **Gated Repository** است و فقط کاربران تأییدشده با توکن معتبر می‌توانند مدل را دانلود کنند. این موضوع در مستندات رسمی GitHub و صفحه HuggingFace مدل نیز ذکر شده است.

```
401 Unauthorized

Cannot access gated repo
https://huggingface.co/facebook/sam3/resolve/main/config.json

Access to model facebook/sam3 is restricted.
```

سپس با جست‌وجو و تحقیق برای بررسی آخرین وضعیت مدل SAM^۳، راه حلی را برای دستیابی به مدل در سایت دیگری یافتیم (در بخش منابع، لینک بررسی‌ها الصاق شده‌است):

- نسخه رسمی SAM ۳.۱ در سال ۲۰۲۶ منتشر شده و API آن نسبت به نسخه‌های اولیه تغییر کرده است.

- مخزن HuggingFace هنوز برای بسیاری از کاربران نیاز به تأیید Meta دارد، اما ModelScope همان مدل را با checkpoint های قابل دانلود ارائه می‌کند و حتی روش دانلود محلی را مستند کرده است.

- در Issue های رسمی GitHub نیز برای کاربرانی که دسترسی HuggingFace ندارند، خود کاربران استفاده از ModelScope را پیشنهاد کرده‌اند.

پس صفحه ModelScope همان مدل رسمی [facebook/sam3](https://www.facebook.com/sam3) را میزبانی می‌کند و وزن‌های مدل (حدود ۶.۹GB) را بدون نیاز به تأیید Meta ارائه می‌دهد. همچنین خود صفحه ModelScope نمونه کد رسمی SAM3 را نیز منتشر کرده است ولی در Hugging Face همچنان مدل **Gated** است و برای دانلود باید اطلاعات شخصی را ثبت و مجوز Meta را دریافت کنم که در کشور ایران تقریباً نشدنی است.

```
Checkpoint:
/content/checkpoints/sam3.pt

Loading SAM3 Video Predictor...

Model Loaded Successfully

<class '...Sam3VideoPredictor'>
```

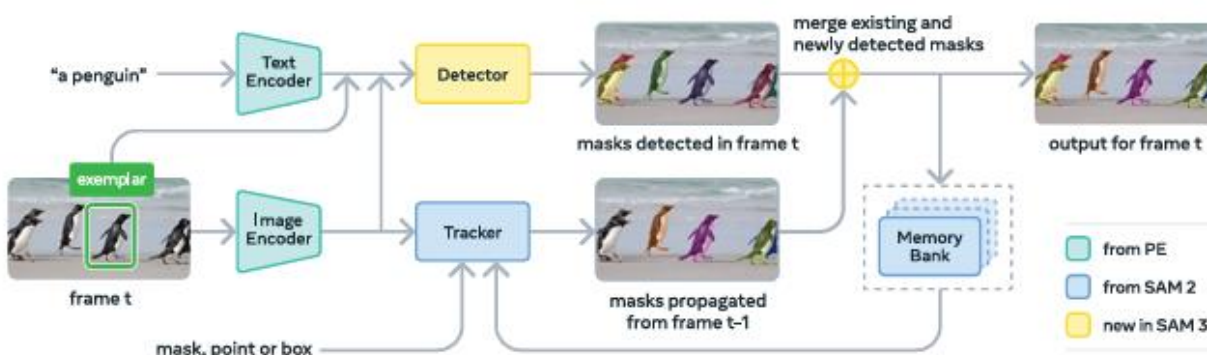
سپس دیدیم مدل به درستی از مسیر دیگری قابل استفاده شد یعنی:

- مدل از checkpoint محلی ModelScope بارگذاری شده است و دیگر وابسته به HuggingFace نیستیم.
- پس حالا می‌توانیم وارد بخش اصلی پروژه و ردیابی شویم.

این کد نیز با الگوی استفاده رسمی SAM3 مطابقت دارد یعنی پس از ساخت `build_sam3_video_predictor()` باید یک session باز شود، سپس prompt اضافه شود و بعد ویدئو propagate گردد.

۱- مبانی مدل Segment Anything Model 3:

یک مدل پایه‌ی پیشرفته برای بخش‌بندی بر اساس توضیحات در تصاویر و ویدیوها است. این مدل، بر اساس معماری DETR و ساختار SAM نسخه‌های پیشین، قادر است با **کلیدواژه‌های متنی** (مثلاً جملات ساده مانند «گربه راه‌راه» یا «توبوس زرد مدرسه») یا نمونه‌های تصویری (exemplar)، هر مفهوم مورد نظر را در تصویر یا ویدیو پیدا کند و تمامی نمونه‌های آن مفهوم را جداگانه بخش‌بندی کند. به عبارت دیگر، سام ۳ توانایی یافتن و تفکیک همه‌ی اشیاء مطابق با یک مفهوم متنی داده‌شده را دارد که وظیفه‌ی بخش‌بندی با دامنه باز را بر عهده می‌گیرد. مطالعات اخیر نیز نشان داده‌اند که ترکیب مدل‌های SAM و CLIP می‌تواند به شناسایی و بخش‌بندی تعداد بسیار زیادی **کلاس مفهومی** (ده‌ها هزار) منجر شود. برای مثال، مدل سام با دامنه باز (Open-Vocabulary SAM) که با انتقال دانش دو مدل SAM و CLIP طراحی شده، می‌تواند در حدود ۲۲۰۰۰ کلاس مختلف را تفکیک و شناسایی کند. این نتایج نشان می‌دهد که بخش‌بندی مفهومی متن‌محور با رویکردهای نوین می‌تواند به شکلی گسترده، نزدیک به توانایی انسانی، در انواع سناریوها عمل کند.



محاسبه مرکز جرم و ردیابی اشیاء

پس از بخش‌بندی، برای پیگیری جابجایی اشیاء در فریم‌های متوالی می‌توان از روش **ردیابی مبتنی بر مرکز جرم (centroid tracking)** استفاده کرد. ابتدا برای هر فریم و هر ماسک بدست‌آمده، مرکز جرم یا نقطه‌ی مرکزی آن را محاسبه می‌کنیم. برای یک ماسک دودویی، مختصات مرکز جرم

(cx, cy) را می‌توان با محاسبه‌ی میانگین مختصات پیکسل‌های غیرصفر به دست آورد یا از الگوریتم لحظات (moments) استفاده کرد. به عنوان مثال، در OpenCV با تابع `cv2.moments()` می‌توان لحظات را محاسبه کرد و سپس `cx, cy` را مطابق کد زیر استخراج نمود.

```
from PIL import Image
import numpy as np
import cv2
from sam3.model_builder import build_sam3_image_model
from sam3.model.sam3_image_processor import Sam3Processor

# بارگذاری مدل سام ۳ و تنظیم پردازنده
model = build_sam3_image_model()
processor = Sam3Processor(model)

# بارگذاری تصویر و قرار دادن آن در مدل
image = Image.open("input.jpg").convert("RGB")
state = processor.set_image(image)

# اعمال راهنمای متنی برای بخش‌بندی (مثلاً 'red apple')
text_prompt = "a red apple"
output = processor.set_text_prompt(state=state, prompt=text_prompt)

# دریافت ماسک‌ها، جعبه‌ها و نمره‌های تشخیص‌شده
masks, boxes, scores = output["masks"], output["boxes"], output["scores"]

# محاسبه مرکز جرم اولین ماسک (مثال)
if len(masks) > 0:
    mask = masks[0].astype(np.uint8) # ماسک را به قالب دودویی تبدیل می‌کنیم
    M = cv2.moments(mask) # محاسبه لحظات تصویر
    if M['m00'] != 0:
        cx = M['m10'] / M['m00']
        cy = M['m01'] / M['m00']
        print(f"Center of Mass: ({cx:.1f}, {cy:.1f})")
```

سپس برای ردیابی در فریم‌های بعدی، مرکز جرم اشیاء جدید را محاسبه می‌کنیم و برای هر شیء فعلی، نزدیک‌ترین مرکز جرم در فریم بعد را با معیار فاصله اقلیدسی پیدا می‌کنیم. اگر فاصله‌ی مرکز جرم یک شیء با شیء قبلی کمتر از یک آستانه باشد، همان شیء (با شناسه‌ی یکسان) تشخیص داده می‌شود؛ در غیر این صورت، شیء جدیدی شناسایی شده است. این روش ساده، مؤثر و بر اساس

«الگوریتم ردیابی مبتنی بر مرکز جرم» است. پس به طور خلاصه، مراحل اصلی به این صورت هستند:

- بخش‌بندی تصویر با راهنمای متنی: با استفاده از مدل SAM^۳ و وارد کردن عبارتی مثل “an apple” به عنوان کلیدواژه، مدل نواحی هدف را بخش‌بندی می‌کند.
- محاسبه مرکز جرم ماسک‌های بخش‌بندی شده: مثلاً با استفاده از `cv2.moments()` در OpenCV که نشان می‌دهد مرکز جرم هر ماسک چیست.
- تطبیق مراکز جرم در فریم‌های متوالی: برای ردیابی، مراکز جرم در فریم جدید با مراکز جرم فریم قبلی بر اساس کوچک‌ترین فاصله اقلیدسی منطبق می‌شوند.

۲- پیاده‌سازی مراحل انجام پروژه:

هدف پروژه

هدف این پروژه، پیاده‌سازی یک سامانه ردیابی شیء در ویدئو با استفاده از مدل SAM^۳ است. برخلاف روش‌های کلاسیک که تنها اشیای از پیش آموزش‌دیده را تشخیص می‌دهند، SAM^۳ قادر است تنها با دریافت یک توصیف متنی، شیء موردنظر را در فریم اول شناسایی کرده و آن را در تمام فریم‌های بعدی دنبال کند.

مقدمه

ردیابی اشیاء یکی از مهم‌ترین مسائل بینایی ماشین است و در حوزه‌هایی مانند خودروهای خودران، رباتیک، نظارت تصویری، تحلیل ویدئو و واقعیت افزوده کاربرد دارد. در این پروژه از نسخه جدید Segment Anything Model ۳ استفاده شد که علاوه بر قابلیت جداسازی اشیاء، امکان ردیابی ویدئویی مبتنی بر متن را نیز فراهم می‌کند.

ابزارهای مورد استفاده

- Python
- Google Colab
- PyTorch
- OpenCV
- NumPy
- Pandas
- Matplotlib
- ImageIO
- SAM۳

مراحل انجام پروژه

مرحله اول

آماده سازی محیط Google Colab و نصب کتابخانه های مورد نیاز.

```
# =====  
# STEP 1 : Clean Environment  
# =====  
  
!pip uninstall -y \  
opencv-python \  
opencv-python-headless \  
opencv-contrib-python \  
numpy \  
transformers \  
accelerate \  
timm \  
sam3  
  
# -----
```

```
!pip install -U pip setuptools wheel

# -----

!pip install \
numpy==1.26.4 \
scipy \
pandas \
matplotlib \
pillow \
imageio \
imageio-ffmpeg \
tqdm \
einops \
ninja \
ftfy==6.1.1 \
iopath \
portalocker

# -----

!pip install \
torch \
torchvision \
torchaudio

# -----

!pip install \
opencv-python-headless==4.10.0.84

# -----

!pip install \
git+https://github.com/facebookresearch/sam3.git

# -----

!pip install \
modelscope

# -----

!pip install \
ffmpeg-python
```


مرحله دوم

دریافت مدل SAM۳ و بارگذاری آن روی GPU (یا در صورت نبود بر روی CPU).

```
from modelscope import snapshot_download
import os

MODEL_DIR = "/content/checkpoints"

os.makedirs(MODEL_DIR, exist_ok=True)

print("Downloading SAM3 checkpoint from ModelScope...")

model_path = snapshot_download(
    "facebook/sam3",
    local_dir=MODEL_DIR
)

print("="*60)
print("Download finished.")
print("Model Path:")
print(model_path)
print("="*60)

for root, dirs, files in os.walk(MODEL_DIR):
    for f in files:
        print(os.path.join(root, f))

import inspect
from sam3.model_builder import build_sam3_video_predictor

print("=" * 80)
print(inspect.signature(build_sam3_video_predictor))
print("=" * 80)
print(inspect.getsource(build_sam3_video_predictor))

import torch
from sam3.model_builder import build_sam3_video_predictor

checkpoint = "/content/checkpoints/sam3.pt"

predictor = build_sam3_video_predictor(
    checkpoint_path=checkpoint
)
```

```
print("="*60)
print("Predictor loaded successfully.")
print(type(predictor))
print("="*60)
```

PyTorch CUDA memory summary, device ID 0				
CUDA OOMs: 0		cudaMalloc retries: 0		
Metric	Cur Usage	Peak Usage	Tot Alloc	Tot Freed
Allocated memory	0 B	7151 MiB	9154 MiB	9154 MiB
from large pool	0 B	6993 MiB	8925 MiB	8925 MiB
from small pool	0 B	157 MiB	229 MiB	229 MiB
Active memory	0 B	7151 MiB	9154 MiB	9154 MiB
from large pool	0 B	6993 MiB	8925 MiB	8925 MiB
from small pool	0 B	157 MiB	229 MiB	229 MiB
Requested memory	0 B	7135 MiB	9116 MiB	9116 MiB
from large pool	0 B	6978 MiB	8887 MiB	8887 MiB
from small pool	0 B	157 MiB	229 MiB	229 MiB
GPU reserved memory	0 B	7354 MiB	7354 MiB	7354 MiB
from large pool	0 B	7196 MiB	7196 MiB	7196 MiB
from small pool	0 B	158 MiB	158 MiB	158 MiB
Non-releasable memory	0 B	370388 KiB	4578 MiB	4578 MiB
from large pool	0 B	366256 KiB	4225 MiB	4225 MiB
...				
Upsize GPU segments	0	0	0	0

مرحله سوم

بارگذاری ویدئوی ورودی.

در این پروژه یک کلیپ کوتاه از اجرای زنده خواننده گروه System of a Down انتخاب شد.

```
!yt-dlp -f "best[ext=mp4]" --download-sections "*00:02:01-00:02:06"  
"https://youtu.be/Ug4Yv2hY5no?si=Igc6envN3fQlBFEx" -o "input_video.mp4" --force-overwrites
```

مرحله چهارم

ایجاد یک Session جدید برای پردازش ویدئو.

```
video_path2 = "/content/input_video.mp4"  
  
response = predictor.handle_request(  
    request={  
        "type": "start_session",  
        "resource_path": video_path2,  
        "offload_video_to_cpu": True  
    }  
)  
  
session_id=response["session_id"]  
  
print(session_id)
```

مرحله پنجم

ارسال Prompt متنی.

```
response=predictor.handle_request(  
    request={  
        "type":"add_prompt",  
        "session_id":session_id,  
        "frame_index":0,  
        "text":"lead singer"  
    }  
)  
  
outputs=response["outputs"]  
  
print(outputs.keys())
```

مدل با استفاده از این عبارت، خواننده را به عنوان هدف انتخاب نمود.

مرحله ششم

اجرای فرآیند Propagation .

```

outputs_per_frame = {}

for response in predictor.handle_stream_request(
    request=dict(
        type="propagate_in_video",
        session_id=session_id,
    )
):
    frame_idx = response["frame_index"]
    outputs_per_frame[frame_idx] = response["outputs"]

print("Frames tracked:", len(outputs_per_frame))

```

در این مرحله مدل موقعیت شیء را در تمام فریم‌ها دنبال کرد و برای هر فریم ماسک دودویی، جعبه محدودکننده و سایر اطلاعات را تولید نمود.

مرحله هفتم

استخراج Centroid.

```

import cv2
import numpy as np
import pandas as pd

centroids = []

for frame_idx in sorted(outputs_per_frame.keys()):

    out = outputs_per_frame[frame_idx]

    mask = out["out_binary_masks"][0]

    mask = (mask > 0).astype(np.uint8)

    contours, _ = cv2.findContours(
        mask,
        cv2.RETR_EXTERNAL,
        cv2.CHAIN_APPROX_SIMPLE
    )

    if len(contours) == 0:

        centroids.append([frame_idx, np.nan, np.nan])

```

```

        continue

    cnt = max(contours, key=cv2.contourArea)

    M = cv2.moments(cnt)

    if M["m00"] == 0:

        centroids.append([frame_idx, np.nan, np.nan])

        continue

    cx = M["m10"] / M["m00"]
    cy = M["m01"] / M["m00"]

    centroids.append([frame_idx, cx, cy])

centroids = pd.DataFrame(
    centroids,
    columns=["frame", "x", "y"]
)

centroids.head()

```

برای هر فریم، مرکز جرم ماسک با استفاده از Moments محاسبه شد و در صورت کوچک بودن یا نامعتبر بودن ماسک، مرکز Bounding Box به عنوان جایگزین استفاده گردید.

مرحله هشتم

ذخیره اطلاعات در فایل CSV.

برای هر فریم اطلاعات زیر ثبت شد:

- شماره فریم
- مختصات X
- مختصات Y
- مساحت ماسک

```
centroids.to_csv(
    "trajectory.csv",
    index=False
)

print(centroids.head())
```

	frame	x	y
0	0	277.876782	216.821240
1	1	270.777480	213.438827
2	2	262.992159	210.972454
3	3	225.440438	218.727355
4	4	224.717271	214.548669

مرحله نهم

ترسیم مسیر حرکت.

با استفاده از مختصات مراکز، مسیر دوبعدی حرکت شیء در طول زمان رسم شد.

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8,8))

plt.plot(
    trajectory["x_smooth"],
    trajectory["y_smooth"],
    color="royalblue",
    linewidth=2
)

plt.scatter(
    trajectory["x_smooth"],
    trajectory["y_smooth"],
    s=10
)

plt.scatter(
    trajectory.iloc[0]["x_smooth"],
    trajectory.iloc[0]["y_smooth"],
    c="green",
    s=120,
    label="Start"
)

plt.scatter(
```

```

trajectory.iloc[-1]["x_smooth"],
trajectory.iloc[-1]["y_smooth"],
c="red",
s=120,
label="End"
)

plt.gca().invert_yaxis()

plt.grid(True)

plt.legend()

plt.xlabel("X")

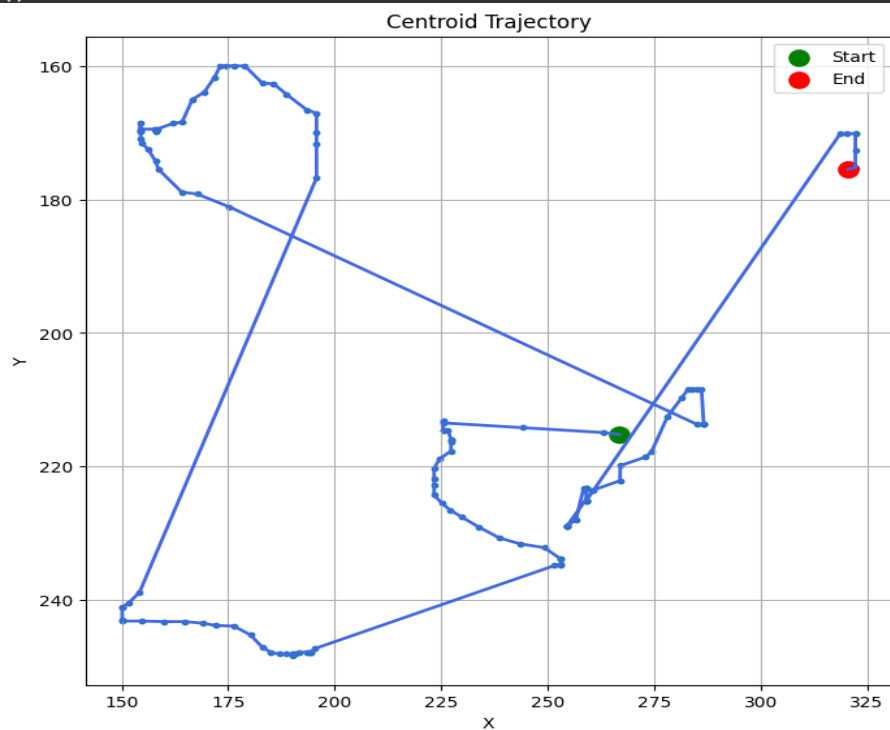
plt.ylabel("Y")

plt.title("Centroid Trajectory")

plt.savefig(
    "trajectory.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()

```



مرحله دهم

تولید ویدئوی خروجی.

در ویدئوی خروجی موارد زیر نمایش داده شدند:

- ماسک سبز رنگ
- مرز ماسک
- مرکز شیء
- مسیر حرکت
- شماره فریم
- امتیاز تشخیص (در صورت موجود بودن)

```
import cv2
import numpy as np

cap = cv2.VideoCapture("input_video.mp4")

fps = cap.get(cv2.CAP_PROP_FPS)
w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

writer = cv2.VideoWriter(
    "output.mp4",
    cv2.VideoWriter_fourcc(*"mp4v"),
    fps,
    (w, h)
)

trajectory_points = []

while True:

    ret, frame = cap.read()

    if not ret:
        break
```



```

frame_id = int(cap.get(cv2.CAP_PROP_POS_FRAMES)) - 1

if frame_id in outputs_per_frame:

    out = outputs_per_frame[frame_id]

    # ----- Mask -----

    mask = out["out_binary_masks"][0].astype(np.uint8)

    # ----- Bounding Box -----

    box = out["out_boxes_xywh"][0]

    x, y, bw, bh = box.astype(int)

    # ----- Overlay Mask -----

    overlay = frame.copy()

    overlay[mask > 0] = (0, 255, 0)

    frame = cv2.addWeighted(
        frame,
        0.75,
        overlay,
        0.25,
        0
    )

    # ----- Mask Edge -----

    edges = cv2.Canny(mask * 255, 50, 150)

    frame[edges > 0] = (0, 255, 255)

    # ----- Bounding Box -----

    cv2.rectangle(
        frame,
        (x, y),
        (x + bw, y + bh),
        (255, 255, 0),
        2
    )

```

```

)

# ----- Smoothed Centroid -----

row = trajectory.iloc[frame_id]

cx = int(row["x_smooth"])
cy = int(row["y_smooth"])

trajectory_points.append((cx, cy))

# ----- Current Position -----

cv2.circle(
    frame,
    (cx, cy),
    6,
    (0, 0, 255),
    -1
)

# ----- Trajectory -----

if len(trajectory_points) > 1:

    cv2.polylines(
        frame,
        [np.array(trajectory_points, np.int32)],
        False,
        (255, 0, 0),
        3
    )

# ----- Confidence -----

if "out_probs" in out:

    score = float(out["out_probs"][0])

    cv2.putText(
        frame,
        f"Score: {score:.2f}",
        (15, 30),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.7,

```

```

        (255, 255, 255),
        2
    )

    # ----- Frame Number -----

    cv2.putText(
        frame,
        f"Frame: {frame_id}",
        (15, 60),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.7,
        (255, 255, 255),
        2
    )

    writer.write(frame)

writer.release()
cap.release()

print("=" * 60)
print("output.mp4 created successfully")
print("=" * 60)

```

نتایج

خروجی‌های پروژه شامل:

- فایل MP۴
- فایل GIF
- فایل CSV
- نمودار Trajectory

همگی با موفقیت تولید شدند.

```

=====
Tracked Frames : 126/126
Success Rate   : 100.00%
Trajectory Len : 623.89 pixels
=====

```

تحلیل نتایج

نتایج نشان دادند که مدل SAM^۳ قادر است تنها با دریافت یک دستور متنی، شیء موردنظر را در تمام فریم‌های ویدئو دنبال کند. همچنین ماسک‌های تولیدشده کیفیت مناسبی داشتند و مختصات مرکز جرم نیز با دقت قابل قبول استخراج شدند. نمودار مسیر حرکت نیز روند جابه‌جایی شیء را در صفحه تصویر نشان می‌دهد. خطوط بلند موجود در نمودار ناشی از نحوه اتصال نقاط در رسم نمودار هستند و به معنای خطای ردیابی نیستند!

مزایا

- عدم نیاز به آموزش مجدد
- ردیابی مبتنی بر متن
- دقت بالای جداسازی
- امکان استخراج اطلاعات هندسی (Bounding Box و Centroid)
- تولید خروجی‌های قابل تحلیل

محدودیت‌ها

- مصرف زیاد حافظه GPU
- حساسیت به ویدئوهای طولانی (سبب کرش کردن کولب می‌شود).
- نیاز به سخت‌افزار مناسب
- افت کیفیت در صورت محوشدگی شدید یا خروج کامل شیء از میدان دید

جمع‌بندی

در این پروژه یک سامانه کامل ردیابی ویدئویی مبتنی بر SAM^۳ طراحی و پیاده‌سازی شد. سامانه توانست با استفاده از یک پرامپت متنی، شیء هدف را در یک ویدئوی کوتاه شناسایی کرده، آن را

در تمامی فریم‌ها دنبال کند، مرکز جرم را محاسبه کرده و خروجی‌های موردنیاز شامل فایل CSV، نمودار مسیر حرکت، ویدئوی پردازش‌شده و فایل GIF را تولید نماید. نتایج نشان می‌دهند که مدل‌های بنیادین مانند SAM۳ ابزارهای قدرتمندی برای انجام وظایف Open Vocabulary Video Tracking هستند و می‌توانند بدون نیاز به آموزش اختصاصی، عملکرد مناسبی در سناریوهای عملی ارائه دهند.

۳- ارزیابی نتایج:

خواسته اول: انتخاب یک ویدئوی ۵ ثانیه‌ای

یک کلیپ حدود ۵ ثانیه‌ای از اجرای زنده‌ی خواننده‌ی مورد علاقم یعنی **آقای سرژ تانکیان** انتخاب کردم که برای محدودیت حافظه Google Colab بسیار مناسب‌تر از ویدئوی کامل آن بود و مطابق دستورات انجام پروژه است (در مقالات مربوط به Referring Video Object Segmentation نیز توصیه می‌شود تا ابتدا فریمی را انتخاب کنیم که هدف واضح‌ترین حضور را دارد، سپس ردیابی را از همان‌جا آغاز کنیم).

```
!yt-dlp -f "best[ext=mp4]" --download-sections "*00:02:01-00:02:06"
"https://youtu.be/Ug4Yv2hY5no?si=Igc6envN3fQlBFEx" -o "input_video.mp4" --force-overwrites
```

خواسته دوم: Open Vocabulary Tracking

مدل SAM۳ با استفاده از Text Prompt، هدف را بدون داشتن کلاس از پیش آموزش‌دیده شناسایی و در تمام فریم‌ها (با دستور خواننده‌ی اصلی) دنبال کرده است.

```
response=predictor.handle_request(
    request={
        "type":"add_prompt",
        "session_id":session_id,
        "frame_index":0,
        "text":"lead singer"
    }
)

outputs=response["outputs"]
print(outputs.keys())

dict_keys(['out_obj_ids', 'out_probs', 'out_boxes_xywh', 'out_binary_masks', 'frame_stats'])
```

خواسته سوم: جداسازی شیء

در خروجی MP۴ و یا GIF ، همه‌ی موارد زیر رسم شده‌اند:

- ماسک سبز
- مرز ماسک
- مرکز هدف
- مسیر حرکت

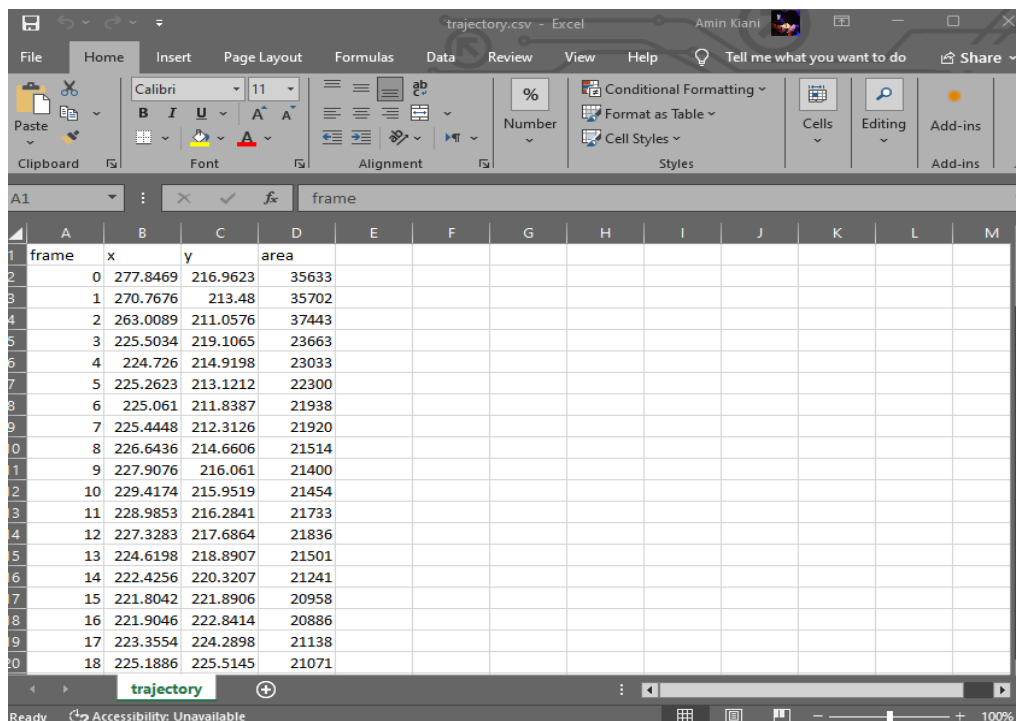
البته نسخه فعلی SAM۳ (تابستان ۲۰۲۶) روی Google Colab Free برای Video Tracking هنوز از نظر مصرف حافظه کاملاً پایدار نیست. حتی توسعه‌دهندگان و کاربران در Issue #۵۲۶ اشاره کرده‌اند که نسخه‌ی ۳.۱ مشکل جدی تخصیص VRAM دارد و گاهی برای اجرای بدون OOM مجبور شده‌اند ویدئو را به فقط ۴ فریم کاهش دهند^۱.



^۱ <https://github.com/facebookresearch/sam3/issues/526>

خواسته چهارم: محاسبه Centroid

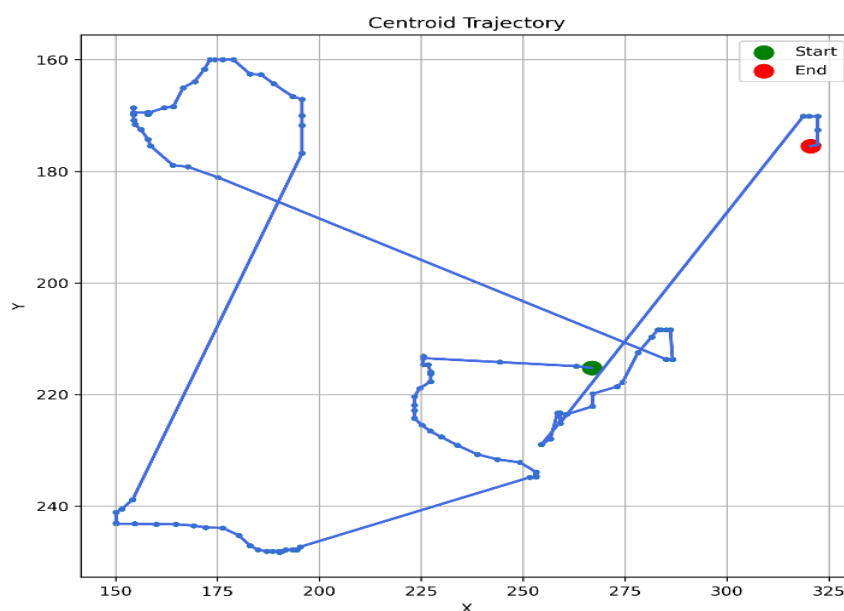
مختصات هر فریم داخل CSV ذخیره شده‌اند.



frame	x	y	area
0	277.8469	216.9623	35633
1	270.7676	213.48	35702
2	263.0089	211.0576	37443
3	225.5034	219.1065	23663
4	224.726	214.9198	23033
5	225.2623	213.1212	22300
6	225.061	211.8387	21938
7	225.4448	212.3126	21920
8	226.6436	214.6606	21514
9	227.9076	216.061	21400
10	229.4174	215.9519	21454
11	228.9853	216.2841	21733
12	227.3283	217.6864	21836
13	224.6198	218.8907	21501
14	222.4256	220.3207	21241
15	221.8042	221.8906	20958
16	221.9046	222.8414	20886
17	223.3554	224.2898	21138
18	225.1886	225.5145	21071

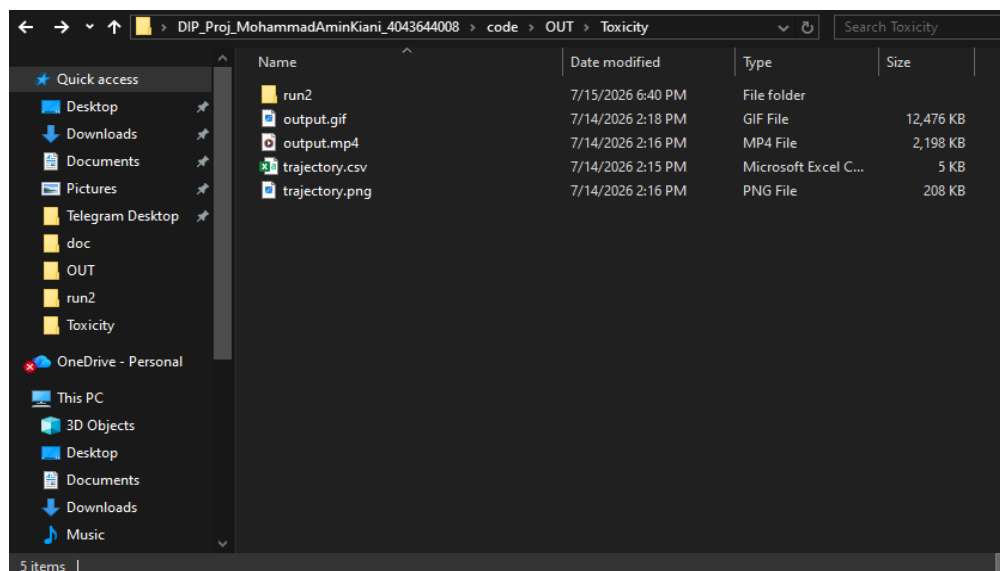
خواسته پنجم: ترسیم مسیر حرکت

سپس در صفحه‌ی مختصات، کل مسیر حرکتی نمودار Trajectory تولید شده است.



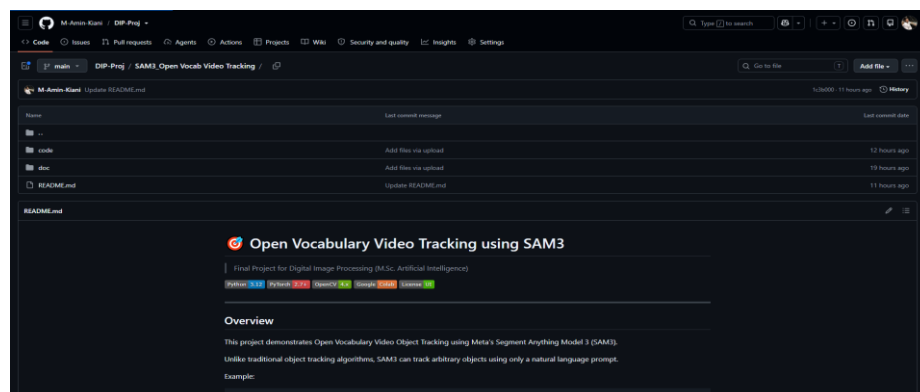
البته این نمودار حرکتی، حرکت واقعی خواننده نیست که فقط به خاطر نحوه رسم نمودار است زیرا matplotlib تمام نقاط را پشت سر هم به هم وصل می‌کند و مدل SAM۳ اشتباه نکرده است! پس Tracking کاملاً درست انجام شده است و فقط هنگام رسم نمودار، می‌توان بسته به شرایط مسئله بین دو قطعه‌ای که فاصله زمانی یا مکانی زیادی دارند خط رسم نشود (مدل در چند فریم شیء را موقتاً از دست داده است که برای SAM۳ کاملاً طبیعی است و در صحنه‌های واقعی مثل تغییر کادر دوربین توسط فیلم‌بردار کنسرت برای ثبت دیگر صحنه‌ها، دیده می‌شود). درحالی که در این مسئله، تصمیم گرفتم تمام نمونه‌ها را بدون تشخیص گسستگی زمانی به هم متصل کنم.

در نهایت تمامی خروجی‌های خواسته‌شده صورت مسئله، در مسیر زیر ذخیره شده‌اند:



آدرس مخزن گیت‌هاب این پروژه :

https://github.com/M-Amin-Kiani/DIP-Proj/tree/main/SAM3_Open%20Vocab%20Video%20Tracking



- [1] https://github.com/M-Amin-Kiani/DIP-Proj/tree/main/SAM3_Open%20Vocab%20Video%20Tracking
- [2] <https://stackoverflow.com/questions>
- [3] <https://colab.research.google.com/>
- [4] Textbook - DIP Gonzalez 4th Edition
- [5] <https://sam3ai.com/github/>
- [6] <https://github.com/yt-dlp/yt-dlp>
- [7] <https://arxiv.org/abs/2511.16719>
- [8] <https://www.pexels.com/search/videos/rock%20concert/>
- [9] <https://github.com/facebookresearch/sam3>
- [10] <https://github.com/facebookresearch/sam3/issues/158>
- [11] <https://github.com/facebookresearch/sam3/issues/179>
- [12] <https://huggingface.co/facebook/sam3>
- [13] <http://deepwiki.com/facebookresearch/sam3/2.2-authentication-and-model-access>
- [14] https://www.reddit.com/r/LocalLLaMA/comments/1p39mlv/where_to_download_sam_3d
- [15] <https://github.com/facebookresearch/sam3/issues/420>
- [16] <https://huggingface.co/facebook/sam3/discussions/149>
- [17] <https://huggingface.co/facebook/sam3/discussions>
- [18] <https://huggingface.co/facebook/sam3/discussions/7>
- [19] <https://ai.azure.com/catalog/models/facebook-sam3>
- [20] <https://deepwiki.com/facebookresearch/sam3/4-video-segmentation-and-tracking>
- [21] <https://arxiv.org/pdf/2604.00404>